



Faculdade de Tecnologia de Sorocaba
Tecnologia em Análise e Desenvolvimento de Sistemas

NICOLLY LEMOS DA SILVA

ANÁLISE DE PIPELINES DE CI/CD

**SOROCABA/SP
2026**

SUMÁRIO

1 PRIMEIRO REPOSITÓRIO.....	3
2 SEGUNDO REPOSITÓRIO.....	4
3 TERCEIRO REPOSITÓRIO.....	5

1 PRIMEIRO REPOSITÓRIO

Repositório: <https://github.com/godotengine/godot>

A infraestrutura de CI/CD do projeto Godot é separada em três pipelines, o `runner.yml` não compila nada, só administra as outras etapas e cancela execuções antigas quando chega um novo push ou pull request. É disparado por push, pull_request, merge_group ou manualmente.

O `static_checks.yml` é o primeiro estágio, é rápido, menos de 40 segundos, valida formatação e estilo de código, além de detectar se houve mudança real no código-fonte, o que decidirá se os builds seguem em frente. Não tem gatilho próprio, sempre roda chamado pelo `runner.yml`, e nas execuções analisadas foi sempre o primeiro a terminar.

Já o `linux_builds.yml` é a pipeline mais pesada, com uma matriz de sete variações de compilação (compiladores e checadores de memória diferentes), usa cache pra acelerar e gera os artefatos finais de cada build aprovado. Só roda se houver mudança de código relevante, e na última execução observada ainda tinha 5 de 7 builds concluídos, confirmando ser a etapa mais demorada do processo.

2 SEGUNDO REPOSITÓRIO

Repositório: <https://github.com/home-assistant/core>

A integração contínua do Home Assistant é concentrada na pipeline ci.yaml, uma estrutura em cascata projetada para cobrir rigorosamente uma das maiores suítes de testes automatizados entre os projetos analisados. A execução é dividida em quatro grandes fases sequenciais que começam na preparação e checagens estáticas, passam pelo controle de qualidade do código, entram na matriz de testes e terminam com o envio dos relatórios de cobertura para o Codecov.

O grande destaque deste repositório é a sua matriz de testes automatizados com o pytest, que divide as verificações por diferentes motores de banco de dado e por profundidade de cobertura, gerando dezenas de jobs em paralelo só para essa finalidade. Além dos testes de código, a pipeline inclui auditorias severas de segurança, checagem de licenças e validação de arquivos Docker. Por conta dessa complexidade, as execuções disparadas pelas ramificações de desenvolvimento são mais longas, levando em média mais de 14 minutos.

3 TERCEIRO REPOSITÓRIO

Repositório: <https://github.com/electron/electron>

O repositório do Electron utiliza uma abordagem de integração contínua extremamente robusta e focada em alto desempenho, centralizada no arquivo `build.yml`. Trata-se de um fluxo fortemente modularizado que orquestra cerca de 15 workflows reutilizáveis, operando sobre uma infraestrutura de servidores dedicados de alta performance, contêineres customizados e uso intensivo de volumes de cache compartilhado.

Para otimizar o tempo de processamento, a pipeline aplica filtros inteligentes de arquivos que identificam a natureza do que foi alterado: se a mudança for apenas na documentação ou em perfis específicos, o sistema pula as compilações pesadas de código nativo sem comprometer as verificações de proteção da ramificação. A execução cobre compilações e testes nativos para diversas combinações de plataformas e arquiteturas em Windows, Linux e macOS, incluindo checagens de memória avançadas com o Clang Tidy e AddressSanitiz